

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – ARCHITECTURE / MODELING (DEVSECOPS METHODOLOGY)

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO4 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Develop Complete DevSecOps Architecture, Methodology Documentation and GitHub Submission.		
Student Name:	Syed Saniya Iqbal	Reg. No.:	192465043
Section Batch:	CSE-Cyber Security	Date:	25/08/2026

Instructions to Students

- Develop a complete and original DevSecOps architecture showing the end-to-end secure software delivery workflow.
- Include development, source control, CI/CD, automated security testing, artifact management, deployment, monitoring and feedback stages.
- Document the methodology clearly, including security controls, tools, workflow relationships and key design decisions.
- Use clear labels, consistent symbols, proper alignment and professionally formatted architecture diagrams.

Submit the editable architecture source files, methodology documentation and an accessible GitHub repository containing all required artifacts.

Question Type	Focus Area	Questions	Marks
ARCHITECTURE MODELING	DevSecOps Methodology	Q1	Q1 – 100
GRAND TOTAL:		100 Marks	

Secure DevSecOps Architecture for Continuous and Secure Software Delivery

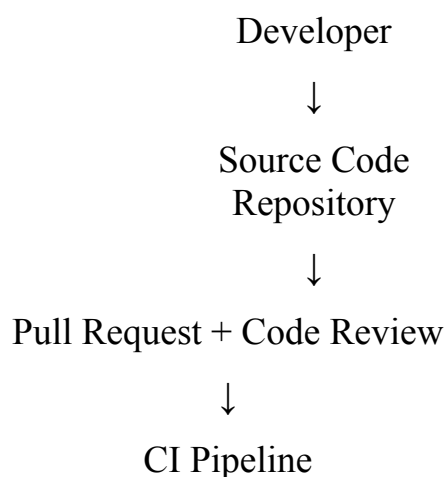
Objective

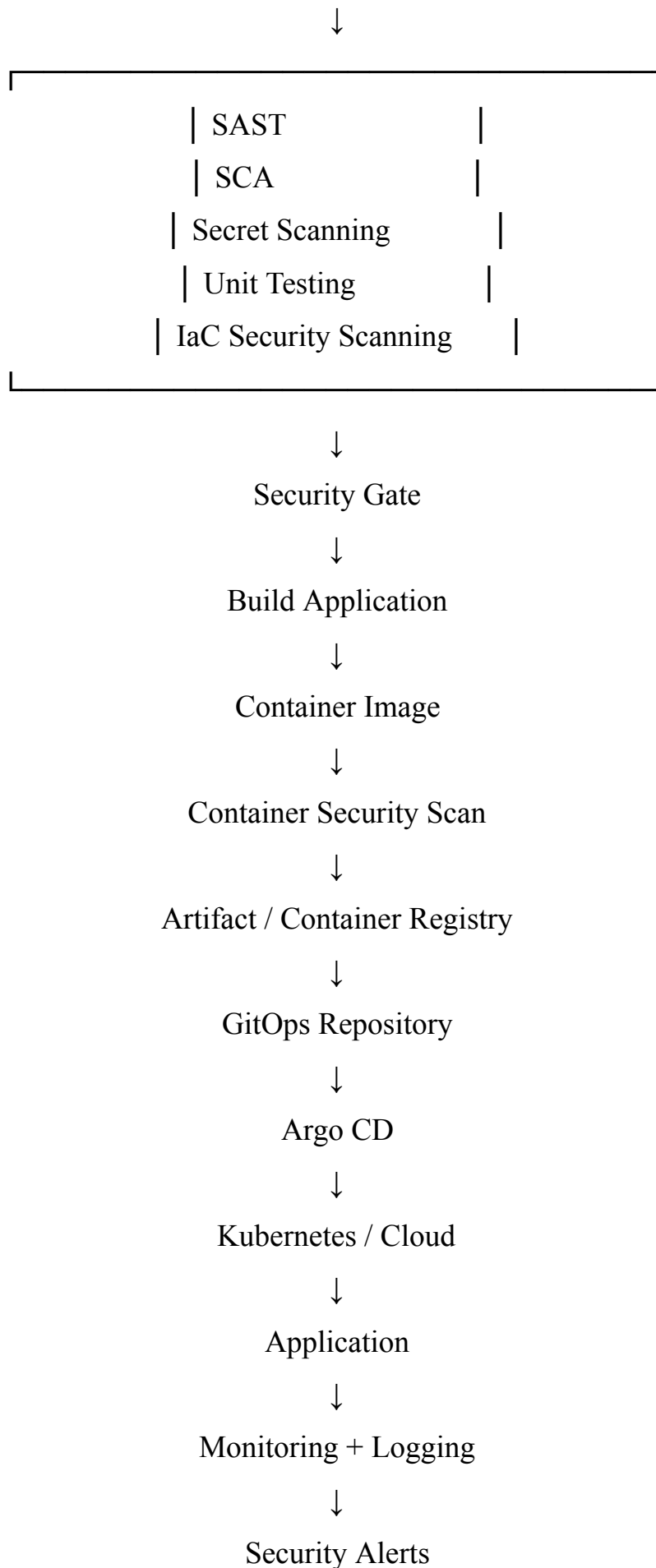
The objective is to design a complete DevSecOps architecture that integrates development, source control, CI/CD, automated security testing, artifact management, deployment, monitoring, and security feedback into one continuous workflow.

The architecture follows the principle of “security at every stage” rather than checking security only after deployment.

Proposed DevSecOps Architecture

End-to-End Workflow

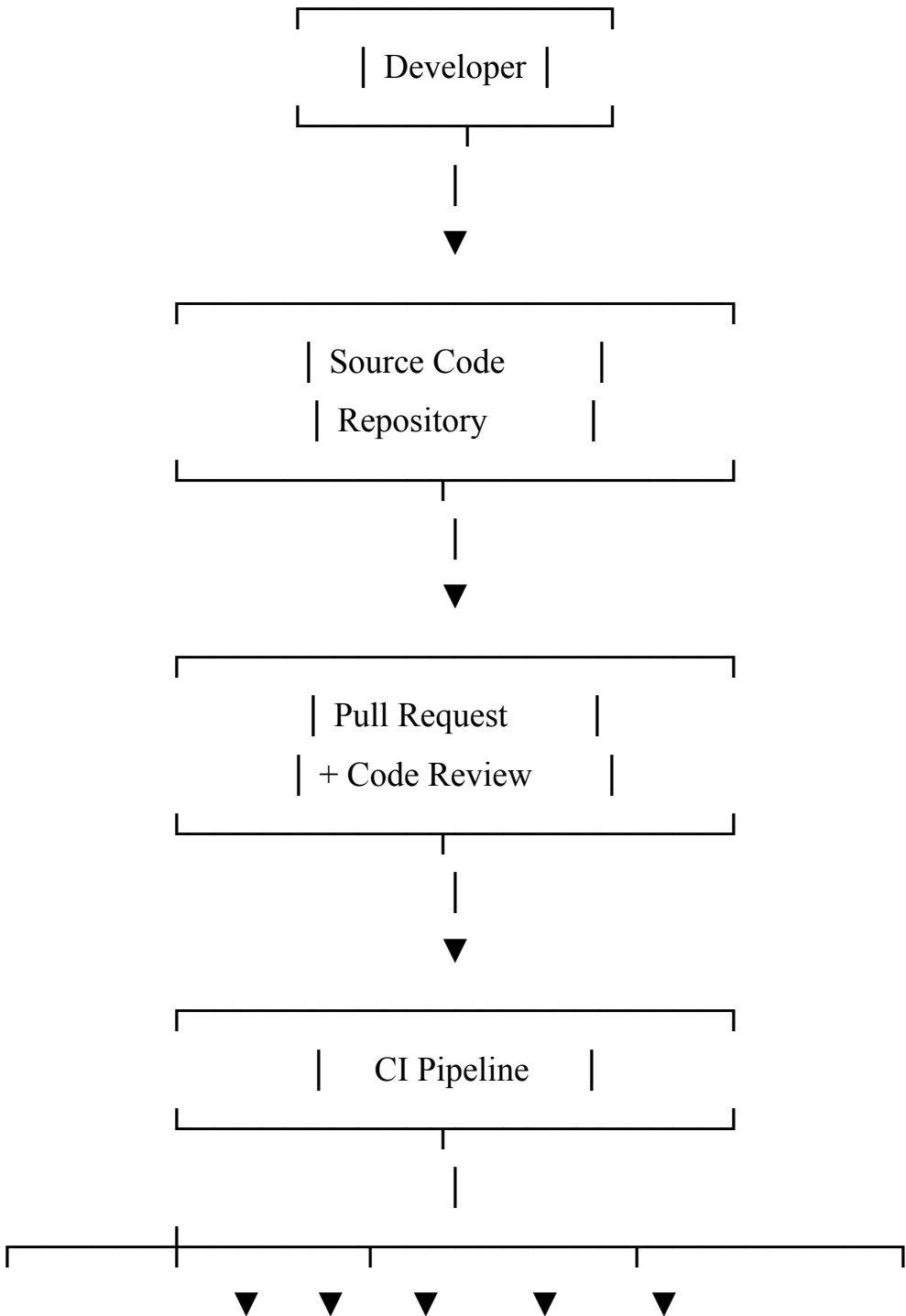


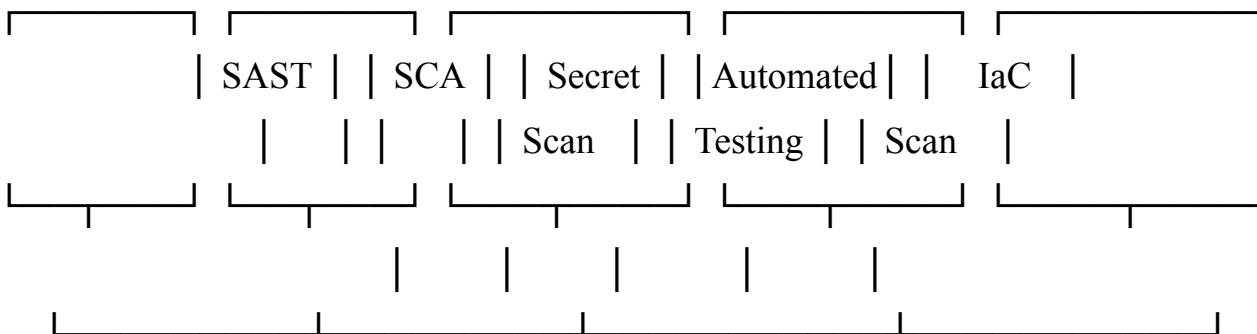


↓
Feedback to Development

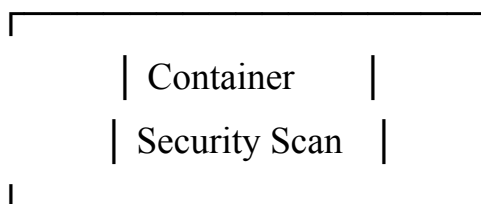
Architecture Diagram

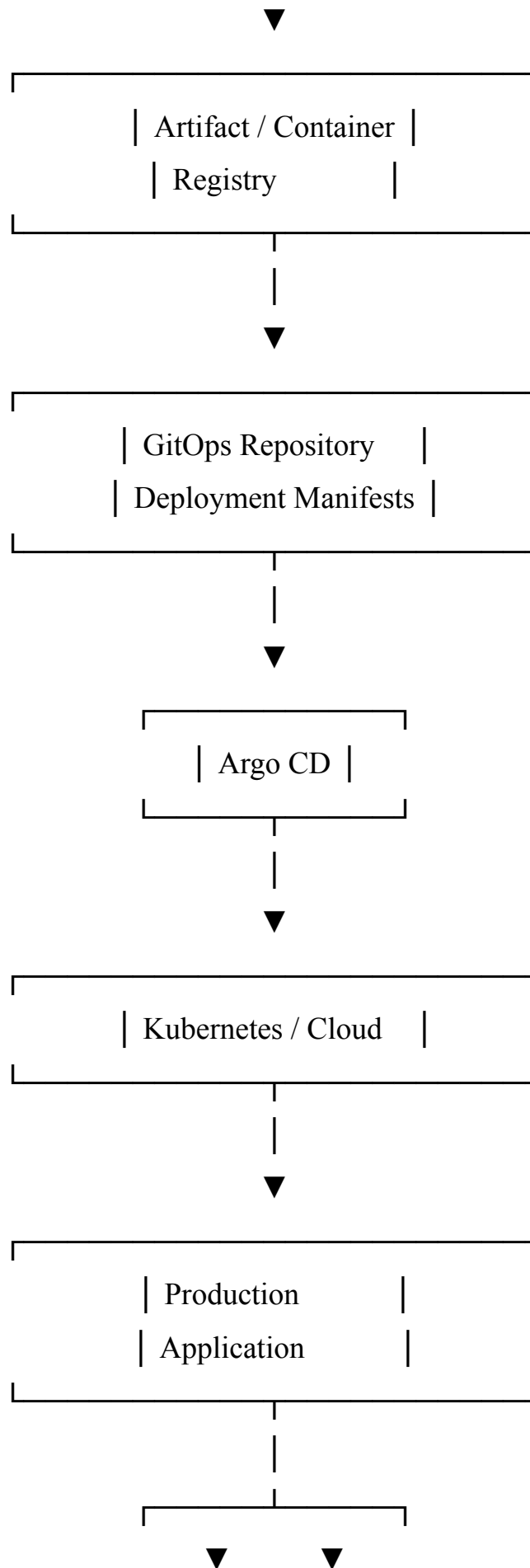
Use this **DiagrammeR** code to create the main architecture diagram.
COMPLETE SECURE DEVSECOPS ARCHITECTURE

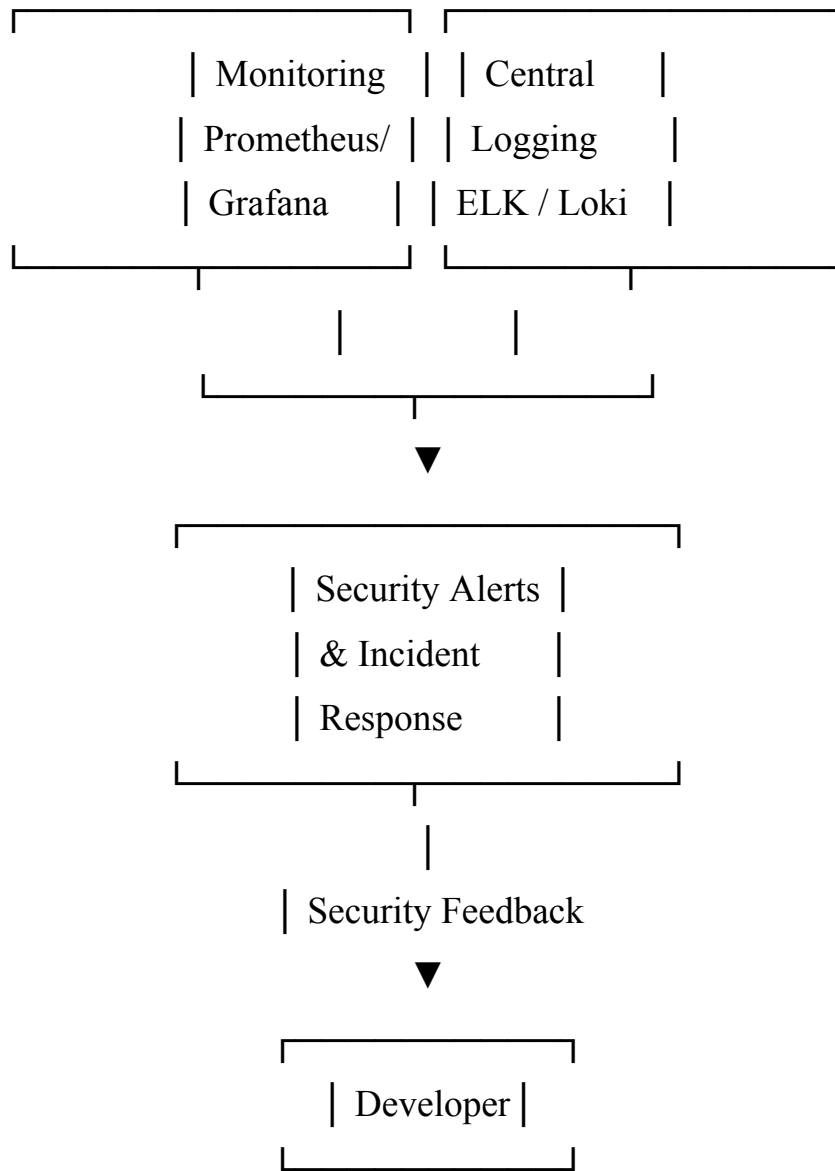




PASS







Explanation of

Architecture 1 Developer

The developer creates and modifies application source code.

Security controls:

- Secure coding practices
- Local testing
- Credential protection
- Code review

Example 1: Developer creates a login module.

Example 2: Developer fixes an SQL injection vulnerability.

2 Source Code Repository

GitHub/GitLab can be used to store source code.

The repository provides:

- Version control
- Branch management
- Pull requests
- Code history
- Access control

Direct modification of the production branch should be restricted.

3 Pull Request and Code Review

Developers submit changes through pull requests.

Another authorized developer reviews the code before it enters the main branch.

Security benefit

It prevents a single developer from directly pushing unreviewed code into production.

CI Pipeline

The CI pipeline automatically executes security and quality checks whenever code changes.

The pipeline contains:

1. SAST
2. SCA
3. Secret scanning
4. Automated testing
5. IaC scanning
6. Build
7. Container scanning

SAST – Static Application Security Testing

SAST analyzes source code without executing the application.

Detects

- SQL injection
- Cross-site scripting
- Hard-coded credentials

- Unsafe functions
- Poor security practices

Example

Developer

↓

C

o

d

e

↓

S

A

S

T

↓

Critical vulnerability

↓

P

i

p

e

l

i

n

e

S

T

O

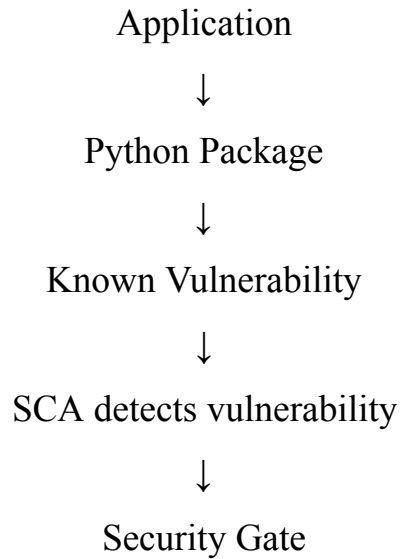
P

The developer must fix the vulnerability before continuing.

SCA – Software Composition Analysis

SCA checks third-party libraries and dependencies.

For example:



Example 1: An outdated Java library contains a known vulnerability.

Example 2: A Python package has a critical CVE.

The pipeline should prevent critical vulnerable dependencies from reaching production.

Secret Scanning

Secret scanning detects accidentally exposed credentials.

It can detect:

- API keys
- Passwords
- Access tokens
- Cloud credentials
- Private keys

Example

API_KEY = "xxxxxxxxx"



S

e

c

r

e

t

S

c

a

n

n

e

r



Secret Detected



P

i

p

e

l

i

n	i
e	l
F	e
a	d

Secrets should instead be stored in a dedicated secrets manager.

Automated Testing

Automated tests verify application functionality.

Testing may include:

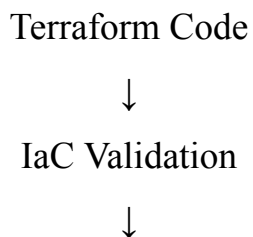
- Unit testing
- Integration testing
- API testing
- Security testing

A build should continue only when the required tests pass.

IaC Security

Infrastructure is managed using **Infrastructure as Code**, such as Terraform.

Example:



IaC Security Scan



Terraform
Plan



Security checks

- Public storage
- Excessive IAM permissions
- Open network ports
- Missing encryption
- Insecure cloud configuration

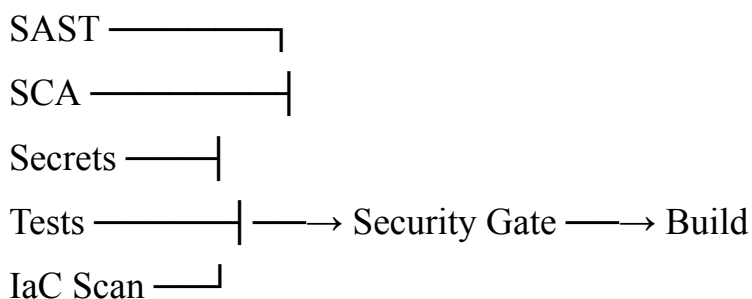
Security Gate

App
p
p
r
o
v
a
l



Infrastructure
Deployment

The security gate is one of the most important parts of the architecture.



If a critical security problem exists, the pipeline stops.

Example

Critical vulnerability → BLOCK

No critical vulnerability → CONTINUE

This prevents insecure software from being deployed.

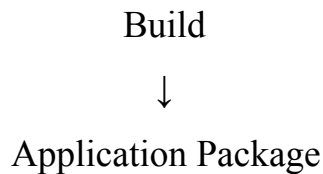
Application Build

After successful security checks, the application is built.

For example:

Source Code

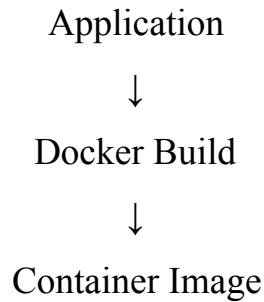




The build process should be repeatable and automated.

Containerization

The application can be packaged into a container.

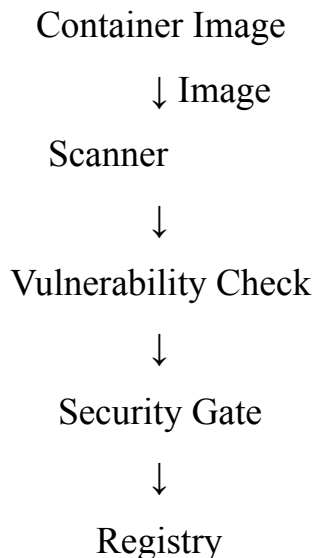


The image should use:

- Minimal base image
- Non-root user
- Updated dependencies
- No hard-coded secrets
- Secure configuration

Container Security Scan

The container image is scanned before being stored or deployed.



The scanner checks operating-system packages and application dependencies.

Artifact / Container Registry

Approved build artifacts are stored in a registry.

Examples:

- Container images
- Application packages
- Build artifacts

The registry should have:

- Access control
- Authentication
- Encryption
- Version management
- Audit logging

GitOps Repository

The GitOps repository stores deployment configuration.

Example:

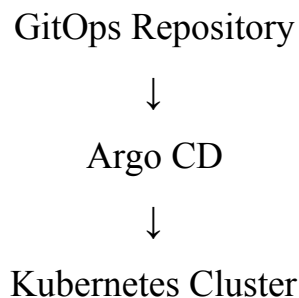
gitops/

```
|
|— deployment.yaml
|— service.yaml
|— config.yaml
|— ingress.yaml
```

Git becomes the **source of truth** for deployment configuration.

Argo CD

Argo CD continuously monitors the GitOps repository.



When the approved deployment configuration changes, Argo CD synchronizes the Kubernetes environment.

Kubernetes / Cloud Deployment

The application is deployed into the production environment.

Security controls include:

- RBAC
- Network policies
- Pod security
- Container restrictions
- Secrets management
- Resource limits
- Audit logging

Monitoring

The deployed application is continuously monitored.

Possible tools:

- Prometheus
- Grafana
- ELK Stack
- Loki

Monitoring collects:

- CPU usage
- Memory usage
- Application errors
- Request information
- Security events
- Failed authentication attempts

Security Feedback

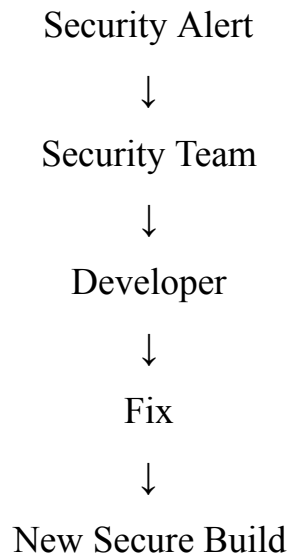
Monitoring produces security alerts.

Production



Monitoring





This creates a continuous improvement loop.

DevSecOps Methodology

The methodology follows these stages:

Stage 1 – Plan

Identify:

- Application requirements
- Security requirements
- Assets
- Threats
- Compliance requirements

Stage 2 – Develop

Develop secure application code using secure coding practices.

Stage 3 – Commit

Push source code to the Git repository.

Stage 4 – Verify

Run:

- SAST
- SCA
- Secret scanning
- Automated testing
- IaC scanning

Stage 5 – Build

Create the application and container image.

Stage 6 – Scan

Perform container vulnerability scanning.

Stage 7 – Store

Store approved artifacts in the registry.

Stage 8 – Deploy

Use GitOps and Argo CD to deploy to Kubernetes.

Stage 9 – Monitor

Monitor application performance and security.

Stage 10 – Respond

Investigate alerts and fix identified problems.

Security Tools

Security Area	Suggested Tool
Source Control	GitHub
CI/CD	GitHub Actions
SAST	Semgrep
SCA	OWASP Dependency-Check
Secret Scanning	Gitleaks
IaC Scanning	Checkov
Container Scanning	Trivy
IaC	Terraform
GitOps	Argo CD
Container Platform	Kubernetes
Monitoring	Prometheus
Visualization	Grafana
Logging	ELK / Loki

Security Controls

Stage	Security Control
Development	Secure coding
Git	Branch protection
Pull Request	Code review
CI	SAST
Dependencies	SCA
Credentials	Secret scanning
IaC	IaC security scanning
Build	Secure build process
Container	Image scanning
Registry	Access control
Deployment	GitOps
Kubernetes	RBAC
Network	Network policies
Runtime	Monitoring
Logging	Centralized logs
Response	Security alerts

Key Design Decisions

Decision 1 – Git-based source control

Git is selected because it provides version history, review and traceability.

Decision 2 – Automated security

Security checks are integrated into CI rather than being performed manually at the end.

Decision 3 – GitOps

Deployment configuration is maintained in Git to provide controlled and auditable deployment.

Decision 4 – Kubernetes

Kubernetes provides container orchestration and scalable deployment.

Decision 5 – Least privilege

Every user and service receives only the permissions required for its task.

Decision 6 – Continuous monitoring

Security does not stop after deployment. Runtime monitoring continuously identifies suspicious behavior.

Repository Structure

Create the GitHub repository like this:

devsecops-security-project/

```
|
|— README.md
|
|— architecture/
|   |— devsecops-architecture.R
|   |— deployment-workflow.R
|   |— iam-model.R
|   |— iac-workflow.R
|   └─ gitops-workflow.R
|
|— application/
|   |— src/
|   └─ tests/
|
|— terraform/
|   |— main.tf
|   |— variables.tf
|   |— outputs.tf
|   └─ security.tf
|
|— kubernetes/
|   |— deployment.yaml
|   |— service.yaml
|   └─ network-policy.yaml
|
```

```
|— gitops/
|   |— deployment.yaml
|
|— security/
|   |— sast/
|   |— sca/
|   |— secrets/
|   |— iac/
|   |— container/
|
|— monitoring/
|   |— prometheus/
|   |— grafana/
|
|— documentation/
|   |— methodology.md
|   |— security-controls.md
|   |— design-decisions.md
|
|— screenshots/
```

GitHub README Content

Your README.md should contain:

Secure DevSecOps Architecture

Project Objective

Design and implement a secure DevSecOps workflow integrating development, CI/CD, security testing, artifact management, GitOps deployment and continuous monitoring.

Architecture

Developer → GitHub → CI/CD → Security Testing
→ Build → Container Scan → Registry
→ GitOps → Argo CD → Kubernetes
→ Monitoring → Security Feedback

Security Tools

- SAST
- SCA
- Gitleaks
- Checkov
- Trivy

Infrastructure

- Terraform
- Kubernetes
- Argo CD

Monitoring

- Prometheus
- Grafana
- ELK/Loki

Security Principles

- Least Privilege

- Defense in Depth
- Secure by Design
- Shift Left Security
- Continuous

Monitoring ##

Repository Structure

Explain the purpose of each project directory.

Methodology

Explain the complete DevSecOps workflow.

Conclusion

The architecture integrates security throughout the software delivery lifecycle.

Validation and Testing

The architecture should be validated with security scenarios.

Test 1 – Vulnerable Code

Test 2 – Exposed Secret

Vulnerable
Code



S
A
S
T



C
r
i
t
i
c
a
l
F
i
n
d
i
n
g



S
e
c
u
r
i
t
y

y
G
a
t
e



Deplo
yment
BLO
CKE
D

Test 3 – Vulnerable Container

Test 4 – Unauthorized Access

Expected Outcomes

After completing the project:

API Key
committed



G

i

t

l

e

a

k

s



Secret detected



Pipeline
BLOCKED

Container Image



T

r

i

v

y



C

r

i

t

i

c

a

l

C

V

E



Registry/Deployment
Blocked

Unauthorized User



- Source code is securely managed.
- Security testing is automated.
- Vulnerable dependencies are detected.
- Secrets are detected before deployment.
- Infrastructure is managed through IaC.
- Containers are scanned.
- Artifacts are securely stored.

IAM / RBAC



Permissions

Groups

Users

Roles

Groups

Users

Groups

Users

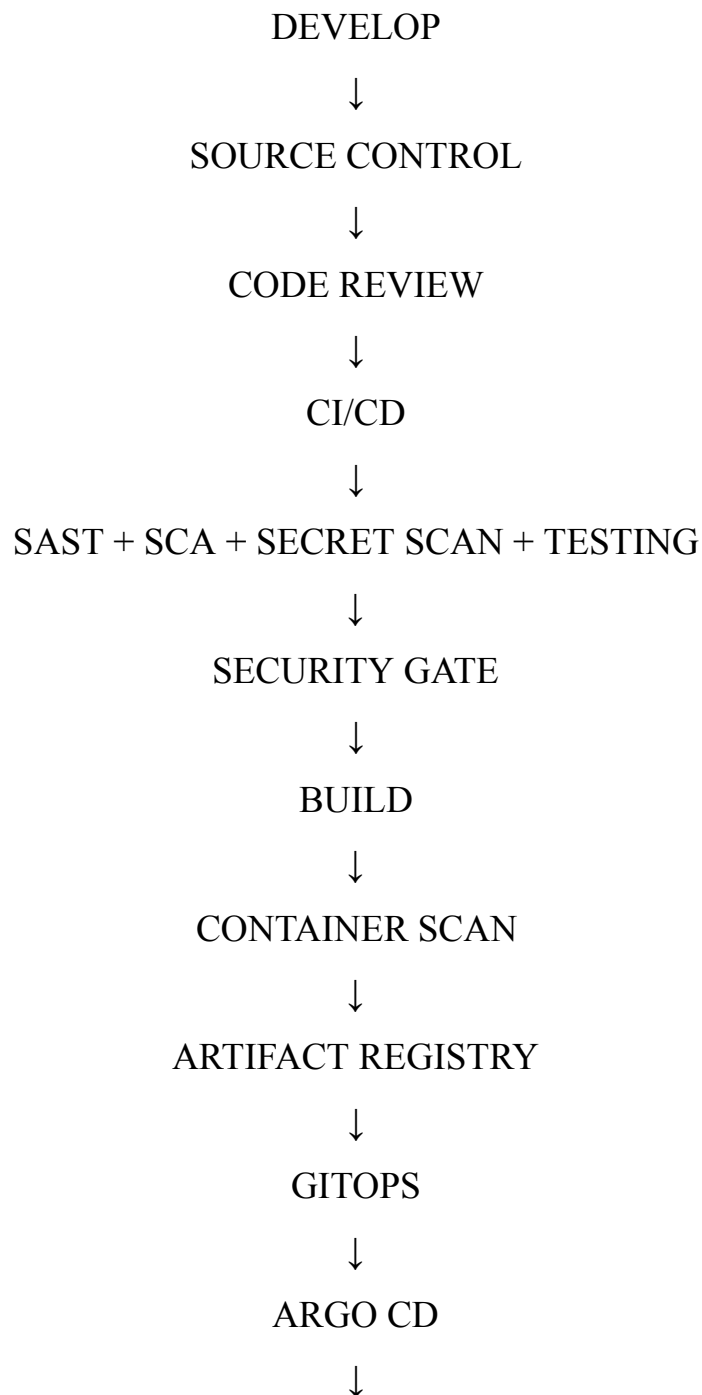
Groups

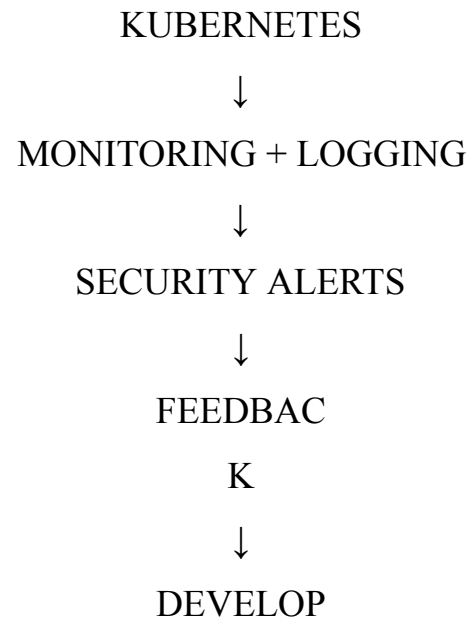
- Deployment is controlled using GitOps.
- Kubernetes access is controlled through RBAC.
- Applications are continuously monitored.
- Security alerts provide feedback to developers.

Conclusion

The proposed **Secure DevSecOps Architecture** provides an end-to-end secure software delivery lifecycle.

The complete flow is:





This design directly satisfies the assignment requirements for development, source control, CI/CD, automated security testing, artifact management, deployment, monitoring, feedback, methodology documentation, editable architecture sources, and GitHub submission.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Complete DevSecOps Architecture Design	20%	Develops a comprehensive and technically accurate DevSecOps architecture integrating development, security, CI/CD, infrastructure and operations components	Develops a clear DevSecOps architecture containing most required components with only minor technical or integration gaps	Develops a basic DevSecOps architecture with limited integration or several missing components	Provides an incomplete or technically inaccurate DevSecOps architecture with weak component integration

Security Integration Across DevSecOps Lifecycle	30%	Effectively integrates security controls and automated security testing across planning, coding, build, test, deployment and operations stages	Integrates appropriate security controls across most DevSecOps stages with only minor omissions	Shows basic security integration but several lifecycle stages or security controls are missing	Shows limited or incorrect security integration across the DevSecOps lifecycle
Methodology and Workflow Documentation	20%	Clearly documents the complete DevSecOps methodology, workflow, tools, responsibilities and security checkpoints with strong technical justification	Documents the DevSecOps methodology and workflow clearly with minor gaps in explanation or justification	Provides basic methodology documentation with limited workflow details or technical explanation	Provides incomplete or unclear methodology documentation with weak technical reasoning

GitHub Repository and Submission Quality	20%	Submits a complete, well-organized GitHub repository containing architecture artifacts, documentation and required files with clear structure and version history	Submits a properly organized GitHub repository containing most required artifacts with minor structural or documentation issues	Submits a partially organized repository with missing artifacts, limited documentation or inconsistent file structure	GitHub submission is incomplete, poorly organized or missing essential architecture and methodology artifacts
Technical Clarity and Professional Presentation	10%	Presents the DevSecOps architecture, methodology and repository professionally with clear relationships, consistent terminology and strong technical clarity	Presents the work clearly and professionally with only minor clarity or formatting issues	Presents the work with basic clarity but noticeable inconsistencies or limited technical explanation	Presents the work poorly with unclear relationships, inconsistent terminology or insufficient technical explanation

Rubrics:

Criteria	Weightage	Q5 (10)	Total Marks (50)
Complete DevSecOps Architecture Design	25%		
Security Integration Across DevSecOps Lifecycle	30%		
Methodology and Workflow Documentation	20%		
GitHub Repository and Submission Quality	15%		

Technical Clarity and Professional Presentation	10%		
---	-----	--	--

Total per Question	100 Marks		
---------------------------	------------------	--	--